

FPT UNIVERSITY

**A PRIVACY-PRESERVING
GRAPH-HYBRID RAG FOR
RELATIONSHIP-AWARE
CODEBASE UNDERSTANDING**

Author: TRAN TIEN DUAT

ID: MSE23196

Supervisor: PhD. DOAN XUAN HUY MINH

Ho Chi Minh City, 2026

Abstract

The emergence of Large Language Models (LLMs) has revolutionized software development, yet challenges such as intellectual property risks and the loss of structural context in code analysis remain significant barriers for enterprise deployment. This study proposes a **Privacy-Preserving Graph-Hybrid RAG Framework** to address the limitations of semantic-only retrieval in codebase understanding. By integrating Abstract Syntax Tree (AST)-based Property Graphs with Vector Databases, the system enables relationship-aware retrieval and enhances multi-hop reasoning across complex dependency chains. The framework is optimized for local inference using hardware acceleration, ensuring source code remains within the local environment while maintaining sub-second latency for structural architectural queries.

Table of Contents

Abstract	2
1 Introduction and Background	4
1.1 Research Problem	4
1.2 Research Objectives	4
1.3 Research Questions	4
1.4 Scope of the Study	5
2 Literature Review and Gap Analysis	6
2.1 Modern RAG Architectures	6
2.2 Knowledge Graph Construction	6
2.3 Gap Analysis: Multi-hop Reasoning Deficit	6
3 Proposed Solution and Research Methodology	7
3.1 System Architecture	7
3.2 Technology Stack	7
3.3 Experimental Dataset	7
4 Implementation and Evaluation Plan	9
4.1 Main Development Stages	9
4.2 Evaluation Metrics	9
4.2.1 Retrieval Metrics	9
4.2.2 Generation Quality (RAGAS)	9
5 Conclusion	10
References	11

1. Introduction and Background

1.1 Research Problem

The integration of LLMs into software development has introduced transformative capabilities for code analysis. However, standard Retrieval-Augmented Generation (RAG) implementations face three critical bottlenecks:

- **Loss of Structural Context:** Codebases are non-linear and highly structured. Standard vector-based RAG treats code as isolated text chunks, failing to capture explicit relationships such as inheritance hierarchies and call graphs [1].
- **Inability for Multi-hop Reasoning:** Traditional retrieval struggle with queries requiring a global view of the codebase’s topology, such as tracing variable impacts across multiple modules [2].
- **Privacy and Intellectual Property Risks:** Enterprise codebases contain sensitive intellectual property. Reliance on cloud APIs poses security risks and compliance barriers regarding data exfiltration [5].

1.2 Research Objectives

- **RO1:** Design a specialized **Automated Knowledge Graph Construction (Auto-KGC)** pipeline integrating Abstract Syntax Trees (AST) and LLM-based extraction to map code entities.
- **RO2:** Implement a **privacy-preserving inference layer** utilizing local hardware acceleration to ensure sensitive code remains within the local environment.
- **RO3:** Develop a **Hybrid Retrieval Engine** fusing Graph Traversal and Vector Search via Reciprocal Rank Fusion (RRF) for multi-hop architectural queries.

1.3 Research Questions

1. How can LLM-based extraction be combined with static analysis (AST) to construct high-fidelity KGs from source code without manual schema definition?
2. Does the integration of graph-based structural context improve the “faithfulness” of codebase-specific answers compared to baseline Vector RAG?

3. Can a local Graph-Hybrid RAG maintain sub-second latency while processing large-scale code indices on hardware-accelerated platforms?

1.4 Scope of the Study

- **In-Scope:** Modular, local-first framework development; dual-path retrieval (Graph and Vector stores); AST-assisted triplet extraction; web-based structural visualization.
- **Out-of-Scope:** Support for non-programming data; domain-specific fine-tuning for non-technical fields; deployment on public cloud infrastructures.

2. Literature Review and Gap Analysis

2.1 Modern RAG Architectures

Research indicates that vector-only systems struggle with global summarization and mapping complex relationships within source code [2]. The evolution toward Modular RAG allows for structured traversal and comprehensive context synthesis.

2.2 Knowledge Graph Construction

Modern Auto-KGC for code utilizes AST-based parsing and LLM triplet extraction. Entity Resolution remains a challenge, requiring semantic similarity and deterministic algorithms to maintain graph canonicalization [3].

2.3 Gap Analysis: Multi-hop Reasoning Deficit

Vector search is limited to point-to-point semantic proximity. There is a clear technical gap for a localized framework that provide relationship-aware retrieval for navigating complex architectural and dependency structures.

3. Proposed Solution and Research Methodology

3.1 System Architecture

The framework utilizes a decoupled architecture to ensure scalability:

- **Data Ingestion:** `CodeSplitter` (AST-based) and `SemanticSplitter` for multi-source parsing.
- **Hybrid Retrieval:** Parallel execution of Vector Search (`Qdrant`) and Graph Traversal (`FalkorDB` via `Cypher`).
- **Context Fusion:** Merging results via Reciprocal Rank Fusion (RRF).

3.2 Technology Stack

Table 3.1: Proposed Technology Stack

Component	Technology	Rationale
Inference Engine	Ollama / MLX	Optimized for local hardware-accelerated inference.
Orchestration	LlamaIndex	Robust support for codebase <code>PropertyGraphIndex</code> .
Graph Store	FalkorDB	High-performance C-based engine with <code>Cypher</code> support.
Vector Store	Qdrant	Advanced metadata filtering and high-concurrency.

3.3 Experimental Dataset

To evaluate the framework’s effectiveness in relationship-aware retrieval and multi-hop reasoning, the study utilizes a selection of high-complexity codebases and standardized benchmarks:

- **Selection Criteria:** The target codebases are selected based on their structural complexity, specifically requiring:
 - **High Modularity:** Projects with extensive inter-module dependencies and complex import structures.
 - **Deep Hierarchies:** Object-oriented codebases with multi-level class inheritance and dense function call graphs to test multi-hop traversal.

- **Primary Source Codebases:** The framework is tested on large-scale, open-source Python and Java repositories from GitHub, such as:
 - *LangChain* or *LlamaIndex*: Chosen for their intricate abstraction layers and dependency on diverse external modules.
 - *Transformers (Hugging Face)*: Selected for its massive codebase size and complex structural relationships between components.
- **Standardized Benchmark:** The **CodeSearchNet** dataset (specifically the Python and Java subsets) is employed as a baseline. This dataset provides a ground-truth mapping between natural language queries and code snippets, allowing for a quantitative evaluation of:
 - **Retrieval Precision and Recall:** Using standard metrics like MRR and Hit Rate.
 - **Graph Fidelity:** Measuring the accuracy of the AST-based triplet extraction against known structural patterns.
- **Privacy-Preserving Acquisition:** In accordance with the localized architecture, all datasets are mirrored to a local environment. All indexing processes, including AST parsing and embedding generation, are performed on local hardware to ensure that no source code or structural metadata is transmitted to external cloud services.

4. Implementation and Evaluation Plan

4.1 Main Development Stages

1. **Weeks 1-2:** Environment setup and AST-based data ingestion pipeline.
2. **Weeks 3-5:** Auto-KGC implementation and Entity Resolution pipeline.
3. **Weeks 6-8:** Hybrid Retrieval Engine and RRF integration.
4. **Weeks 9-12:** API development, visualization interface, and system optimization.

4.2 Evaluation Metrics

4.2.1 Retrieval Metrics

- **Hit Rate & MRR:** Quality of context ranking for code entities.
- **Context Precision:** Signal-to-noise ratio in multi-hop dependency retrieval.

4.2.2 Generation Quality (RAGAS)

- **Faithfulness:** Correctness of answers based on local codebase context.
- **Answer Relevance:** Semantic alignment with architectural queries.

5. Conclusion

This project establishes a standardized, privacy-preserving pipeline for relationship-aware codebase retrieval. By bridging the gap between unstructured vector space and structured AST-based topology, the framework provides a foundation for advanced multi-hop reasoning in localized enterprise environments.

References

- [1] A. Hogan *et al.*, “Knowledge graphs,” *ACM Computing Surveys*, 2021.
- [2] D. Edge *et al.*, “From local to global: A graphrag approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [3] G. Papadakis *et al.*, “Comparative analysis of approximate string matching algorithms for entity resolution,” *IEEE TKDE*, 2020.
- [4] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *NeurIPS*, 2020.
- [5] N. F. Liu *et al.*, “Lost in the middle: How language models use long contexts,” *arXiv preprint arXiv:2307.03172*, 2023.